

Exercício 00

Basicamente, um compilador traduz todas as suas linhas de código para outra linguagem – normalmente, uma de alto nível para outra de baixo nível (Assembly ou linguagem de máquina). Já o interpretador basicamente faz esse trabalho de conversão aos poucos, sempre que uma declaração ou função é executada, por exemplo. MATLAB, Lisp, Perl e PHP são apontadas como interpretadas.

Exercício 01

Análise Léxica: É a primeira fase do processo de compilação, também é conhecida como leitura ou scanning. O objetivo nessa fase é identificar unidades lexicais ou lexemas que compõem o programa. O analisador léxico lê todos os caracteres do programa fonte e verifica se eles pertencem ao alfabeto da linguagem. Caso um caractere não pertença ao alfabeto da linguagem deve ser gerado um erro léxico.

Análise Sintática: A análise sintática tem como objeto validar a gramática do programa, nessa etapa o objetivo é reconhecer se a estrutura gramatical do código fonte está de acordo com as regras sintáticas da linguagem.

Análise semântica: O objetivo dessa etapa é verificar se a semântica do programa fonte tem consistência. Para isso é utilizada a árvore sintática e as informações contidas na tabela de símbolos.

Geração de código intermediário: Nessa fase é gerado uma sequência de código denominada código intermediário, que posteriormente em outras fases irá gerar o código objeto. Por ventura essa fase pode não existir e a compilação pode ser feita diretamente para o código objeto, isso é comum em compiladores auto residentes.

Otimização de código: A otimização de código é a estratégia de examinar o código intermediário, produzido durante a fase de geração de código com objetivo de produzir, através de algumas técnicas, um código que execute com bastante eficiência.

Geração de código final: A geração de código objeto é a última etapa do processo de compilação e recebe como entrada uma representação intermediária que mapeia a linguagem objeto.

Exercício 02

A análise léxica também conhecida como scanner ou leitura é a primeira fase de um processo de compilação e sua função é fazer a leitura do programa fonte, caractere a caractere, agrupar os caracteres em lexemas e produzir uma sequência de símbolos léxicos conhecidos como tokens.

Exemplo:

Código: `a[index] = 4 + 2`

Sequência de tokens: `<id, 1> <[,> <id, 2> <[,> <=,> <numero, 4> <+,> <numero, 2>`

Tabela de símbolos:

Entrada	Informações
---------	-------------

1	a - variável inteira
2	index - variável inteira

Exercício 03

Porquê na análise sintática, é verificado se o programa está de acordo com a gramática da linguagem.

Exercício 04

É efetuado a leitura do programa fonte, caractere a caractere, agrupar os caracteres em lexemas e produzir uma sequência de símbolos léxicos conhecidos como tokens. A partir disso é diferenciado palavras reservadas de identificadores, pois no caso de palavras reservadas é a sequência de caracteres que formam a palavra reservada, no caso de identificadores são os caracteres que formam os nomes das variáveis e funções.

Exercício 05

A análise léxica é muito prematura para identificar alguns erros de compilação, porém um exemplo de possível erro é a presença de caracteres que não pertencem a nenhum padrão conhecido da linguagem, como por exemplo o caractere ϕ .

Exercício 06

Têm como tarefa principal verificar se o programa está de acordo com a GLC e inserir informações de tipo das variáveis na tabela de símbolos.

Exercício 07

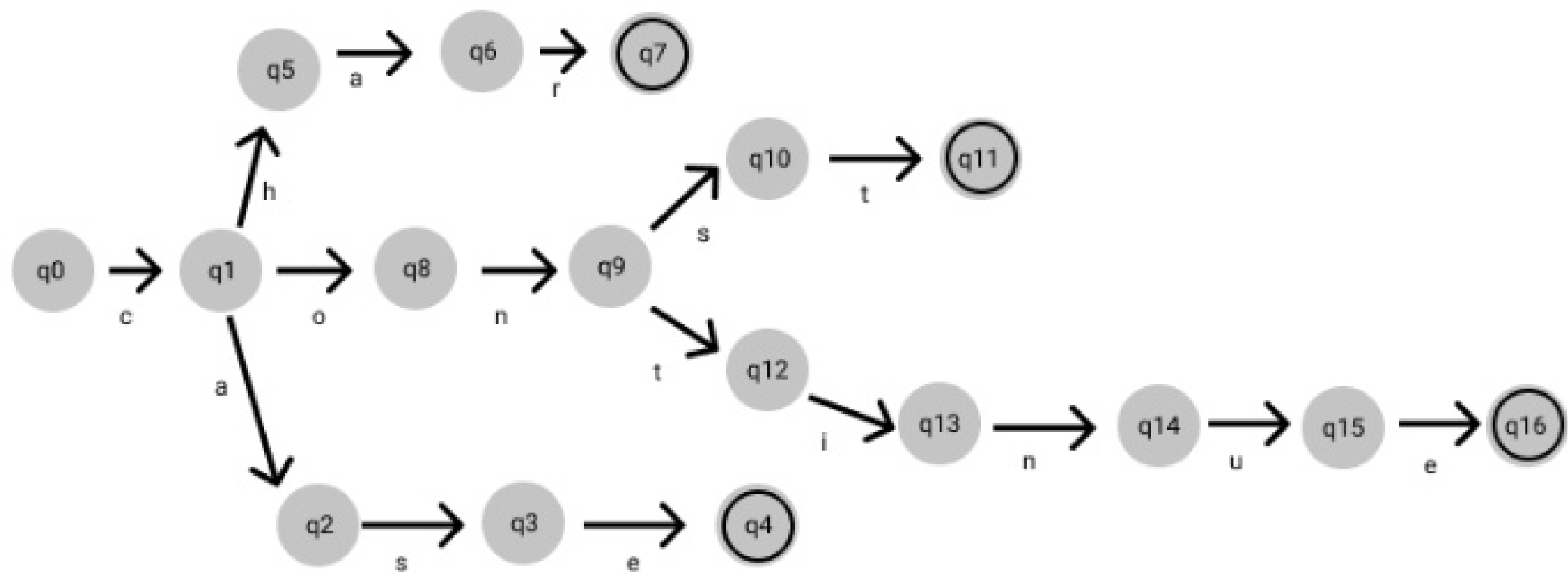
Expressão regular: Uma expressão regular é uma notação para representar padrões em strings. Serve para validar entradas de dados ou fazer busca e extração de informações em textos.

Autômato: Um autômato funciona como um reconhecedor de uma determinada linguagem e serve para modelar uma máquina ou, se quiser, um computador simples.

Tokens: Os tokens são símbolos léxicos reconhecidos através de um padrão.

Símbolos Léxicos: São unidades básicas de texto do programa. Eles são representados internamente por três informações: classe do token, valor do token e posição do token.

Exercício 08



Exercício 09

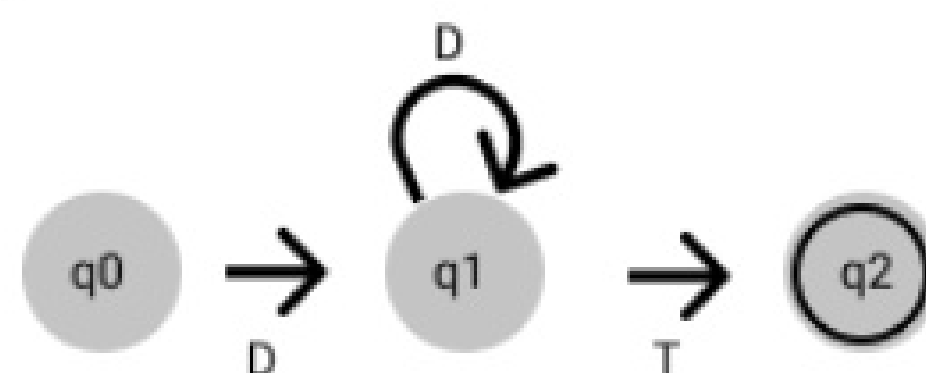
D= [0-9]

L= [a-z, A-Z]

T= [\b, \t, \n, +, -, *, /, >, <, =, |, :, ;, (,), ...]

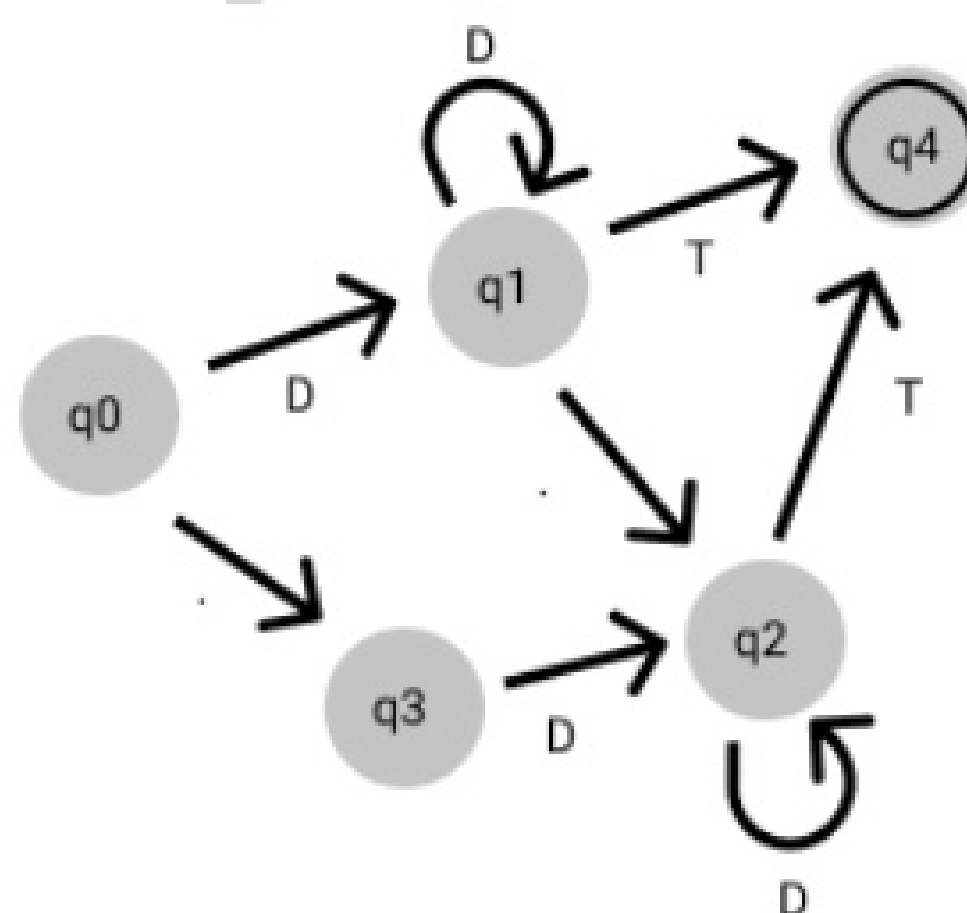
Números inteiros não sinalizados

Expressão Regular: (D^+)



Números reais não sinalizados (sem uso de potências de 10 ou notação científica)

Expressão regular: $(D^+ | D^+ \cdot | _D^+ | D^+ \cdot D^+)$



Operadores aritméticos: +, -, /, *, (,)

Expressão regular: $(+ | - | / | * | " | ')$



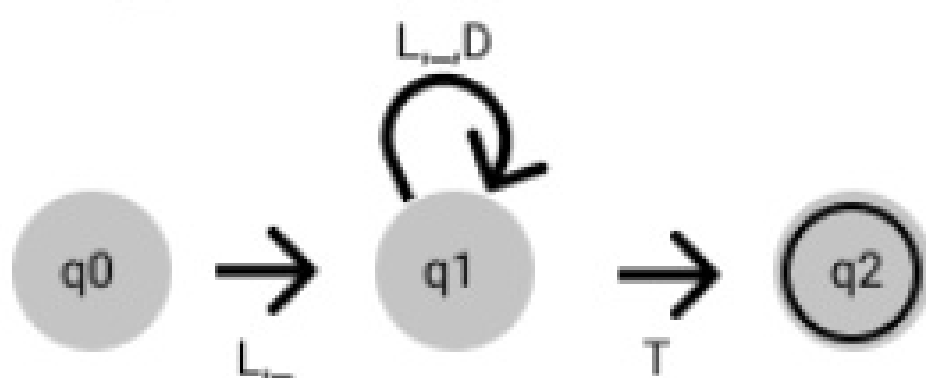
Operador de atribuição: $=$

Expressão regular: $(=)$



Identificadores

Expressão regular: $(L | _) (L | _ | D)^*$



Exercício 10

$(s|S)(e|E)(l|L)(e|E)(c|C)(t|T)$

Exercício 11

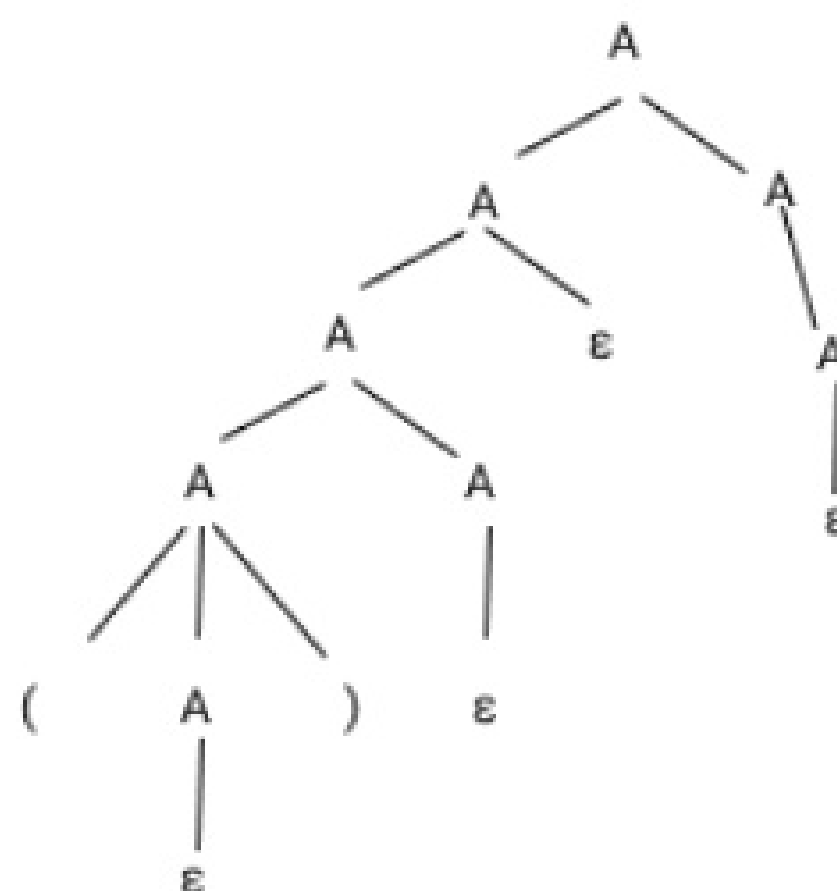
$(/*)((D|L)|"/"|(/ | *)^* (*/),$ onde $D = [a-z, A-Z]$ e $L = [0-9]$.

Exercício 14

$S \rightarrow 0S1S \mid 1S0S \mid \epsilon$

Exercício 15

A linguagem gerada foi “todas as strings com parênteses balanceados”



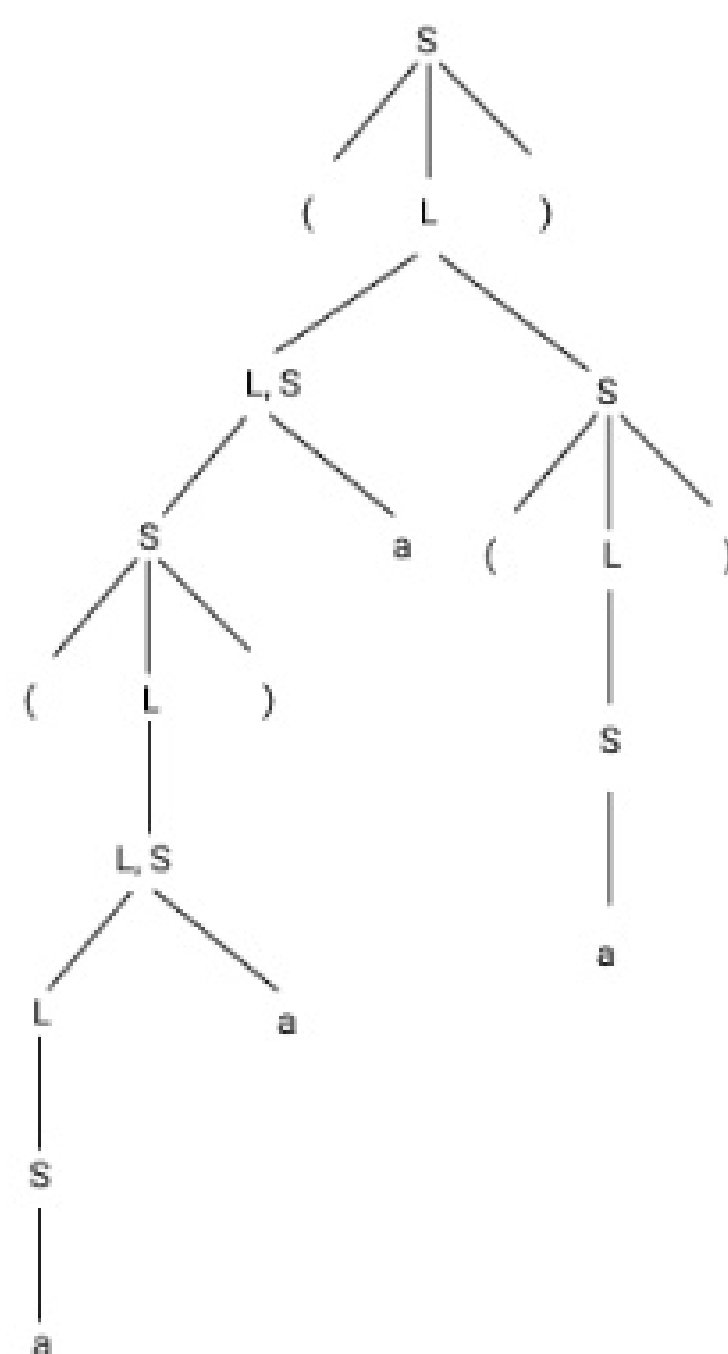
Exercício 16

E
 $E - T$
 $E - F$
 $E - (E)$
 $E - (E + T)$
 $E - (E + T * F)$
 $E - (E + T * 6)$
 $E - (E + F * 6)$
 $E - (E + 5 * 6)$
 $E - (T + 5 * 6)$
 $E - (F + 5 * 6)$
 $E - (4 + 5 * 6)$
 $T - (4 + 5 * 6)$
 $F - (4 + 5 * 6)$
 $3 - (4 + 5 * 6)$

Exercício 17

Exercício 18

- 1) $S = lm \Rightarrow (L) \Rightarrow (L, S) \Rightarrow (L, S, S) \Rightarrow ((S), S, S) \Rightarrow ((L), S, S) \Rightarrow ((L, S), S, S) \Rightarrow ((S, S), S, S) \Rightarrow ((a, S), S, S) \Rightarrow ((a, a), S, S) \Rightarrow ((a, a), a, S) \Rightarrow ((a, a), a, (L)) \Rightarrow ((a, a), a, (S)) \Rightarrow ((a, a), a, (a))$
- 2) $S = rm \Rightarrow (L) \Rightarrow (L, S) \Rightarrow (L, (L)) \Rightarrow (L, (a)) \Rightarrow (L, S, (a)) \Rightarrow (L, a, (a)) \Rightarrow (S, a, (a)) \Rightarrow ((L), a, (a)) \Rightarrow ((L, S), a, (a)) \Rightarrow ((S, S), a, (a)) \Rightarrow ((S, a), a, (a)) \Rightarrow ((a, a), a, (a))$
- 3)



- 4) A gramática se provou não ambígua, pois só foi possível encontrar uma árvore de derivação para a linguagem
- 5) Equivalente com as tuplas em python

Exercício 18.7

Expressões booleanas não ambíguas. A linguagem descreve o conjunto de todas as cadeias de 0s e é da forma x^ny^n , onde x e y são do mesmo comprimento.

Exercício 20

Não existe um fator esquerdo comum.